

Afterquery Write-up – Research Scientist (Post-training)

Eric Ballouz - May 2026

1 Choice of Framework

I used a `Gymnasium` wrapper because of its very general framework, modularity and easy implementation. It allows me to abstract the RL process into:

- Environment creation
- Optimizer choice
- Learning
- Evaluation

`Gymnasium` is especially appropriate given the relatively low-dimensional discrete action space (see §3)

2 Benchmark Selection

I used the `terminalbench2` benchmark. `tau2_bench` evaluates interactive agentic processes, e.g. conversational agents that coordinate with users over multiple turns. `terminalbench2` fits more neatly into the RL framework since its "steps" are independent calls to the agent without user interaction, though as will be explained later, the experiment as designed will measure (and seek to minimize) corrections and interactions with the user.

The `terminalbench2` benchmark still requires long-horizon reasoning, repository navigation, debugging, code modification, dependency management, and iterative validation — all behaviors that are interesting challenges for the agent.

3 Environment Creation

The goal of the RL is to train an agent (in this case Mistral) to complete benchmark tasks. The environment is constructed as follows:

- **State:** a vector of four numbers. The first (0 or 1) indicates whether the run attained the result, i.e. solved the task; this state constitutes a convergence signal. The second is the index of the last action taken by the optimizer from a discrete list of possible actions, and represent the short-term policy memory. The third is the number of guiding prompts from the user. The fourth is the total number of steps (thinking steps and prompts) the agent took, and measures the trajectory length and complexity.
 - **Action space:** the actions are user guiding prompts to the agent:
 - “Inspect the repo structure”
 - “Read test and validation files, then formulate a first step”
 - “Debug the latest output”
 - “Propose a code modification. Modifications to files should be in the format `<cmd> echo "[modified_code]" > [filename] </cmd>`”
 - “Install packages that might help pass validation”
 - “Check if modifications improved the performance”
- . The prompts are fed to the agent, which can operate independently and read the outputs / effects of its own actions.
- **Reward:** $r = f$ where f is the number of successfully completed benchmark tasks. Another option could penalize the number of steps—i.e. outside user interventions— and takes the shape $r = f - 0.01n$, where 0.01 is a penalty factor and n is the number of interventions. Another reward, $r = f - 0.01n_{\text{total}}$ penalizes both outside prompts and the number of agent’s internal thinking steps, favoring efficient, straightforward prompting and reasoning.

The RL policy does not directly optimize token generation. Instead, it learns high-level semantic interventions that steer an autonomous coding agent. Suppose the following sequence of actions is chosen: ”Inspect the repo structure”, ”Propose a code modification[...]” An outer loop iterates through these actions. First, the agent is asked to inspect the repository; it may comment on it or even propose actions or commands. These are executed within an internal loop until the agent stops proposing modifications. Once the agent stops executing tasks—it is ”stuck”—it is fed the second user prompt, initiating another deeper reasoning cycle. The RL environment appends more actions (user prompts) until the agent has been guided sufficiently and completes the task.

This framework creates a hierarchical control structure in which reinforcement learning shapes debugging and reasoning behavior without requiring direct fine-tuning of the language model itself. The framework is relatively easy to implement and low-dimensional, while letting the agent ”shine” (i.e. it preserves the agent’s native behavior and preferences). Re-optimizing the model’s weights is intractable on my laptop.

As described, taking a step in the RL environment involves running `harbor` with a custom agent (`agent.py`) against a set of benchmark tasks. The custom agent—which uses Mistral—is initialized to read user prompts (actions) as part of a conversation. This architecture implies that `harbor` is run at each RL step, which is inefficient but easy to implement and interfaces well between the RL framework and the custom agent.

4 Model Selection and Training

4.1 Model

- I picked Mistral-7B because it performs poorly on certain tasks in comparison with Claude. In particular, it takes much longer to complete `largest-eigenval`, and thus could benefit from well-worded prompts from the user.

4.2 RL Algorithm

I believe GRPO is better suited for optimizing agent trajectories. GRPO generates simultaneous policies and compares them, which is better adapted to an agent that can complete a task in different ways. Specifically, due to agent stochasticity, the optimal GRPO policy will be, on average, better than the alternatives, though it may not generate exactly the same reward at each run. However, GRPO is better suited to distributive implementations that can realize multiple simultaneous trajectories before comparing them. Given my restricted computational resources, I picked PPO. In this case, the action space is discrete, low-dimensional, and easy to integrate with `Gymnasium`.

4.3 Metrics

The agent traces are stored, which records the internal reasoning steps initiated by each user prompt. Moreover, any agent commands or code patches are also stored.